

Apuntes Semana 8 Clase 1

Anthony Fuentes Calvo
Instituto Tecnológico de Costa Rica
Clase (14/04/2026)
IC-6200 Inteligencia Artificial
Profesor: Steven Pacheco Portugués

Resumen—Este documento consolida apuntes del curso de Inteligencia Artificial y un repaso guiado de redes neuronales. Primero se registran, en formato pregunta–respuesta, conceptos de preprocesamiento (normalización Min–Max y estandarización Z-score), pruebas de asociación/independencia (chi-cuadrado y Spearman), la expresión del F_1 -score y definiciones de *data cleaning* y *data reduction*, además de funciones de activación comunes. Luego se sintetizan recomendaciones de redacción para informes académicos (sin código y con figuras explicadas). Finalmente, se desarrolla un repaso sobre clasificación con MNIST y redes densas: codificación *one-hot*, interpretación de capas y neuronas, y la formulación matricial $XW + b$ para computar varias salidas y/o múltiples muestras de manera directa. El documento incluye ejemplos numéricos y diagramas para fijar intuición, manteniendo notación matemática consistente.

I. QUIZ 4

1. Anote la fórmula y explique la diferencia entre normalización y estandarización.

- **Normalización (Min–Max):**

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (1)$$

- **Estandarización (Z-score):**

$$x' = \frac{x - \mu}{\sigma}, \quad (2)$$

donde μ es la media y σ la desviación estándar.

- **Diferencia:** la normalización reescala a un intervalo (p.ej., $[0, 1]$) usando $x_{\min}$ y $x_{\max}$; la estandarización centra en 0 y ajusta la varianza a 1 usando μ y σ .

2. ¿Cuál prueba debo utilizar para probar la independencia de dos variables en caso de que sean categóricas y cuál en caso de que sean numéricas no normales?

- **Categóricas:** prueba **chi-cuadrado de independencia** sobre una tabla de contingencia.
- **Númericas no normales:** **correlación de Spearman** (asociación monótona basada en rangos).

3. Anote la fórmula para calcular el F_1 -score.

$$F_1 = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}}. \quad (3)$$

4. Describa en qué consiste el Data Cleaning y Data Reduction.

- **Data Cleaning:** proceso de mejorar la calidad del dataset eliminando o corrigiendo ruido, inconsistencias y valores faltantes (p.ej., imputación, corrección de formatos, tratamiento de outliers).
- **Data Reduction:** proceso de reducir el volumen o la dimensionalidad de los datos conservando información relevante (p.ej., selección de características, agregación, muestreo o reducción de dimensionalidad).

5. Mencione 2 funciones de activación distintas a la función sigmoide que pueden ser usadas en redes neuronales.

- **ReLU:** $\text{ReLU}(x) = \max(0, x)$.
- **Tangente hiperbólica:** $\tanh(x)$, con salida en $[-1, 1]$.

II. SUGERENCIAS PAPER

En el paper de la tarea, seguir una redacción correcta indicada en el TEC Digital. En el proyecto se espera una buena investigación respecto a bibliografías, un buen estado del arte y una introducción que siga una metodología con todo lo que se va a aplicar para explicar la forma de cómo concretar el objetivo del proyecto o tarea. Las figuras deben ir debidamente referenciadas y con un buen texto de explicación. La tarea debe venir con un buen abstract.

Los resultados deben venir en el reporte para definir adecuadamente los resultados.

Un informe no lleva código; evitar a toda costa agregar código para explicar nada. Preferiblemente, con palabras explicar lo que se está haciendo; no más bloques de código en el informe.

III. REDES NEURONALES

Para llegar a este punto fue necesario ver varios temas importantes para entenderlo mejor.

Regresión lineal y regresión logística (función sigmoide). Partimos de la regresión lineal y, para clasificación, de la regresión logística, donde la función sigmoide transforma la salida lineal en una probabilidad en $[0, 1]$.

Las redes neuronales permiten tener **modelos no lineales** a lo largo del modelo, al componer capas con funciones de activación no lineales.

Clasificación de MNIST. En este contexto se utiliza el dataset MNIST:

- **Originalmente:** imágenes de 128×128 píxeles.
- **Transformadas a:** 28×28 , lo que equivale a 784 *features*.
- **Muestras:** $\sim 60,000$ muestras.
- **Canal:** 1 canal (escala de grises).
- **Clases:** números de 0 a 9.

Este dataset solo tiene un canal; los valores de píxel suelen normalizarse y etiquetarse en un rango de 0 a 1.

Es una **clasificación múltiple** (multiclase), ya no es **binaria**, por lo que es más complejo. **Se resuelve, en un primer paso,** planteando clasificadores binarios (por ejemplo, *one-vs-rest*) antes de abordar la formulación multiclase completa con redes neuronales.

III-1. Título: Regresión Logística (Clasificación Binaria): En la figura 1 se resume el caso binario: combinación lineal $f_{w,b}(x) = wx + b$, función sigmoide g y salida $h(x) = g(f(x))$. Para MNIST, la entrada es un vector de 784 características (aunque el diagrama ilustre un número menor de nodos por claridad).

Tomamos los datos, aplicamos **regresión lineal** y luego **regresión logística**: primero la parte lineal $f_{w,b}(x) = wx + b$ y después la no linealidad (sigmoide) para obtener probabilidades. La figura 2 muestra cómo se codifican las clases 0–9 como vectores *one-hot* en un problema multiclase como MNIST.

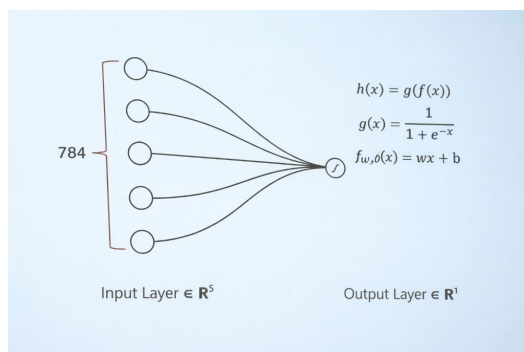


Figura 1. Regresión logística para clasificación binaria: capa de entrada, salida escalar y composición $h(x) = g(f(x))$ con sigmoide.

One-hot vector

class	one-hot codification
0	1 0 0 0 0 0 0 0 0 0
1	0 1 0 0 0 0 0 0 0 0
2	0 0 1 0 0 0 0 0 0 0
3	0 0 1 0 0 0 0 0 0 0
4	0 0 0 0 1 0 0 0 0 0
5	0 0 0 0 1 0 0 0 0 0
6	0 0 0 0 0 1 0 0 0 0
7	0 0 0 0 0 0 1 0 0 0
8	0 0 0 0 0 0 0 1 0 0
9	0 0 0 0 0 0 0 0 1 0

Figura 2. Vectores *one-hot* por clase (codificación binaria con un solo 1 en la posición de la etiqueta).

Tomamos nuestras etiquetas y , para cada una de esas categorías, usamos **codificación one-hot**. Esa clasificación es bastante útil: permite tener **muchas regresiones logísticas** en paralelo (por ejemplo, una salida por clase).

Representación de 1 Feature (píxel)

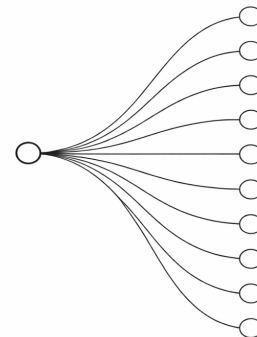


Figura 3. Representación de 1 *feature* (píxel): una entrada conectada a varias salidas (p.ej., una regresión logística por clase).

Si solo se tiene **un píxel** (una sola entrada), se obtiene la arquitectura anterior: cada círculo de salida corresponde a una **regresión logística** y se comparte la misma información de entrada con cada una de esas unidades.

La salida del modelo es la **predicción**; la **etiqueta verdadera** indica los valores correctos. Si la predicción y la etiqueta comparten la misma codificación *one-hot* (los bits activos en las mismas posiciones), la predicción es correcta.

Si ahora hay **dos píxeles** (dos *features* de entrada), cada arista que llega a un nodo tiene un **peso**: la información de cada entrada se combina hacia cada nodo, de modo que en

este caso cada conexión dispone de **dos pesos** (uno por cada entrada).

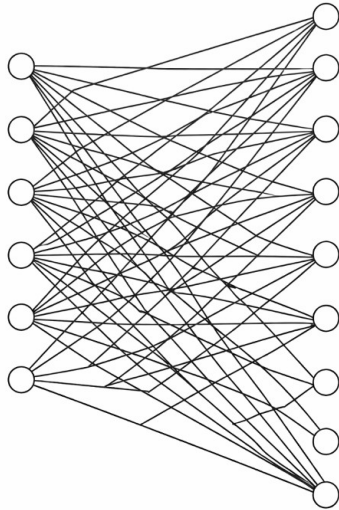


Figura 4. Capa densa (*fully connected*): varias entradas (p.ej., 6 *features*) conectadas a varias salidas (p.ej., 9 neuronas); cada arista corresponde a un peso.

Ahí hay **demasiados pesos** que hay que **calibrar** durante el entrenamiento.

Conversión de vector a matriz.

- En lugar de calcular cada regresión lineal de forma vectorial (una a una).
- Cambiamos los vectores por **matrices** para hacer **una sola operación** y no N .
- Donde N es el **tamaño de la capa siguiente**.
- Se usan de nuevo conceptos de **álgebra lineal**.



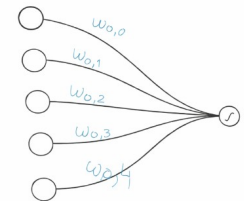
Figura 5. Matriz de pesos W (todas las regresiones): j es el tamaño de la capa siguiente y n el número de *features* (p.ej., 784 entradas en MNIST).

En W , cada **columna** recoge los pesos de una **entrada** (los coeficientes que van desde un *feature* hacia todas las neuronas de la capa siguiente). Si hay que **transformar** esos gráficos (la red como grafo) a notación compacta, ¿qué obtenemos? Se obtiene la **matriz de pesos** W , que resume todas las regresiones en una sola estructura.

A partir de ahí surge la pregunta: ¿cuál debe ser el **tamaño** de esa matriz? La figura 6 plantea el caso con entrada en $\mathbb{R}^5$, salida en $\mathbb{R}^1$ y pesos $w_{0,j}$: el número de **filas** coincide con las neuronas de salida y el de **columnas** con las entradas (regla: si hay n *features* y m salidas, la forma típica es $m \times n$; aquí $m = 1$ y $n = 5$, es decir $W \in \mathbb{R}^{1 \times 5}$).

¿Cuál debería ser el tamaño de la siguiente matriz?

$w_{0,0} \ w_{0,1} \ w_{0,2} \ w_{0,3} \ w_{0,4}$



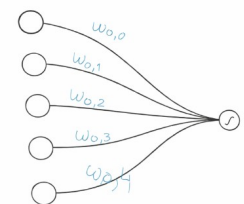
Input Layer $\in \mathbb{R}^5$ Output Layer $\in \mathbb{R}^1$

Figura 6. Representación de una capa densa con 5 entradas y 1 salida para determinar las dimensiones del vector (fila) de pesos W .

La figura 7 plantea de nuevo ¿cuál debe ser el tamaño de la matriz? con entrada $\mathbb{R}^5$, salida $\mathbb{R}^1$ y una fila de pesos $W_{0,0}, \dots, W_{0,4}$, es decir $W \in \mathbb{R}^{1 \times 5}$ (una fila por neurona de salida, una columna por entrada).

¿Cuál debería ser el tamaño de la siguiente matriz?

$w_{0,0} \ w_{0,1} \ w_{0,2} \ w_{0,3} \ w_{0,4}$



Input Layer $\in \mathbb{R}^5$ Output Layer $\in \mathbb{R}^1$

Figura 7. Ejercicio: tamaño de W para capa densa con entrada $\mathbb{R}^5$, salida $\mathbb{R}^1$ y pesos $W_{0,j}$ ($j = 0, \dots, 4$).

Si hay **varias neuronas de salida**, se espera $W \in \mathbb{R}^{m \times n}$

con $m > 1$. La figura 8 relaciona la **arquitectura** (capa densa $5 \rightarrow 2$) con la **notación matricial**: cada fila de W corresponde a una neurona de salida y cada columna a una entrada.

¿Cuál debería ser el tamaño de la siguiente matriz?

$w_{0,0}$	$w_{0,1}$	$w_{0,2}$	$w_{0,3}$	$w_{0,4}$
$w_{1,0}$	$w_{1,1}$	$w_{1,2}$	$w_{1,3}$	$w_{1,4}$

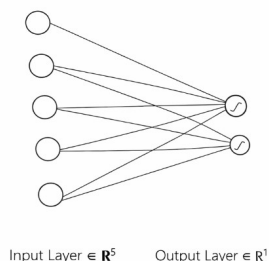


Figura 8. Relación entre la arquitectura de una capa densa (5 entradas, 2 salidas) y la matriz de pesos $W \in \mathbb{R}^{2 \times 5}$ (índices $w_{i,j}$: salida i , entrada j).

En el contexto de MNIST ya citado (784 *features* por imagen), conviene precisar el objeto que sale de la capa: **cada neurona de salida** entrega un **escalar** (su combinación lineal, y luego la activación); al recorrer **toda esa capa**, esos escalares se agrupan en un **vector**. Para almacenar los coeficientes que conectan todas las entradas con todas las neuronas de salida se usa **una matriz como la de la siguiente figura**.

Con diez clases (dígitos $0, \dots, 9$), los índices de fila suelen ser $i = 0, \dots, 9$; la **última fila** se denota $w_{9,0}, \dots, w_{9,784}$ (y no $w_{10,0}$, que implicaría una undécima salida). La figura 9 resume el planteamiento.

¿Cuál debería ser el tamaño de la siguiente matriz?

$w_{0,0}$	$w_{0,1}$	$w_{0,2}$	...	$w_{0,784}$
$w_{1,0}$	$w_{1,1}$	$w_{1,2}$	...	$w_{1,784}$
...	...	...	...	...
$w_{10,0}$	$w_{10,1}$	$w_{10,2}$	...	$w_{10,784}$

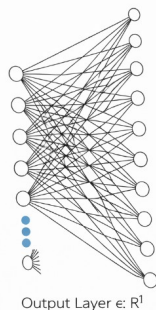


Figura 9. Pregunta sobre el tamaño de W en un esquema tipo MNIST: pesos $w_{i,j}$ con columnas $j = 0, \dots, 784$ y una fila por neurona de salida i . Para 10 clases, $i \in \{0, \dots, 9\}$; la última fila es $w_{9,0}$, no $w_{10,0}$.

En la práctica, una capa densa aplica primero una **transformación afín**: a la entrada (vector de *features* $x_1, \dots, x_n$)

se le multiplica la matriz de pesos W y se suma el vector de **sesgos** b (bias), obteniendo un vector de salida $y_1, \dots, y_j$ antes de pasar por la función de activación. La figura 10 resume ese bloque como $XW + B = Y$ en la notación del diagrama.

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \begin{bmatrix} w_{0,0} & \dots & w_{0,n} \\ \vdots & & \vdots \\ w_{j,0} & \dots & w_{j,n} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_j \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_j \end{bmatrix}$$

Figura 10. Transformación afín de una capa: entrada, matriz de pesos W , suma de sesgos B y vector de salida Y (la no linealidad se aplica después, elemento a elemento).

Cuando hay **dos regresiones de salida** (dos neuronas o dos salidas paralelas), **cada una lleva su propio conjunto de pesos** (y su propio sesgo). Hay que **calcular primero la regresión lineal** $w^\top x + b$ en cada rama y, acto seguido, aplicar la **función sigmoide** a cada resultado escalar para obtener valores en $[0, 1]$. La figura 11 ilustra el procedimiento con un ejemplo numérico.

Ejemplo 2 operaciones

- $x = [3, 4, 5, 6]$
- $W_1 = [3, 2, 4, 5]$
- $b_1 = 2$
- $W_1 \cdot x + b_1 = [3, 4, 5, 6] \cdot [3, 2, 4, 5] + 2 = 67 + 2 = 69$
 $\text{sigmoid}(W_1 \cdot x + b_1)$
- $x = [3, 4, 5, 6]$
- $W_2 = [4, 3, 2, 1]$
- $b_2 = 3$
- $W_2 \cdot x + b_2 = [3, 4, 5, 6] \cdot [4, 3, 2, 1] + 3 = 40 + 3 = 43 = 43$
 $\text{sigmoid}(W_2 \cdot x + b_2)$

Figura 11. Ejemplo con dos operaciones sobre el mismo vector de entrada x : pesos W_1 y W_2 (y sesgos b_1, b_2) distintos; en cada caso $W_i \cdot x + b_i$ y luego $\text{sigmoid}(\cdot)$.

Cuando el mismo cálculo se plantea en **forma matricial**, se puede hacer **todo de una sola vez**: un único producto que agrupa las filas de W y el vector de sesgos, en lugar de repetir $w^\top x + b$ salida por salida. Así el resultado sale de forma **más**

directa y compacta. La figura 12 muestra el mismo ejemplo anterior, pero con x , W (filas = regresiones o salidas, columnas = *features*), b y la forma $xW^T + b$ seguida de $\text{sigmoid}(\cdot)$ aplicada vectorialmente.

Mismo ejemplo pero con matrices

- $x = [3, 4, 5, 6]$
- $W = [[3, 2, 4, 5], [4, 3, 2, 1]]$, (*regresion, feature*)
- $b = [2, 3]$
- $xW^T + b$
- $\text{sigmoid}(xW^T + b)$
- $x = [3, 4, 5, 6]$
- $W_2 = [4, 3, 2, 1]$
- $b_2 = 3$
- $\text{sigmoid}(xW^T + b)$

Figura 12. Mismo ejemplo en notación matricial: $xW^T + b$ y luego $\text{sigmoid}(\cdot)$ elemento a elemento (dos filas en W para dos salidas).

El producto x (vector fila) por la matriz de pesos (en columnas, una por salida) más el sesgo b se puede escribir de forma explícita; el resultado lineal antes de la sigmoide es el vector fila de la figura 13.

$$\begin{pmatrix} 3 & 4 & 5 & 6 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 \\ 2 & 3 \\ 4 & 2 \\ 5 & 1 \end{pmatrix} + \begin{pmatrix} 2 & 3 \end{pmatrix}$$

Solución

$$\begin{pmatrix} 69 & 43 \end{pmatrix}$$

Figura 13. Desarrollo numérico del mismo caso: producto matricial y suma de b ; solución lineal $[69, 43]$ (cada componente coincide con $W_i \cdot x + b_i$ del ejemplo separado).

Para pasar de un solo vector de entrada a **varias instancias a la vez** (varios *samples* o un mini-lote), X se organiza como **matriz**: una fila por ejemplo. Con la matriz de pesos W (sin contar el sesgo por separado) y el vector b con un sesgo por regresión, la transformación lineal compacta es $XW + b$. La figura 14 recoge las definiciones.

- X , Representa una matriz de input, una fila por instancia. Permite computar múltiples samples a la vez
- W , Matriz de pesos a excepción del bias, una fila para cada regresión y una columna por cantidad de regresión
- b , Contiene un bias por cada regresión.
- $XW + b$

Figura 14. Notación matricial: X como matriz de entradas (una fila por instancia), W matriz de pesos, b sesgos por salida y cálculo $XW + b$ para procesar múltiples muestras en paralelo.

IV. TEMA NUEVO

En esta ocasión se vuelve a la **clasificación binaria**. La red produce valores en el intervalo $[0, 1]$ (p.ej., tras la sigmoide en la salida), interpretables como probabilidad de la clase positiva.

Hay una **entrada** (por ejemplo, características o píxeles); esa entrada se procesa con **diez regresiones logísticas** en paralelo (capa oculta). Las diez salidas intermedias se **colapsan** después en **una sola regresión logística** final, cuya salida corresponde a la etiqueta binaria (convencionalmente 0 o 1, o un par *one-hot* como $[1, 0]$ o $[0, 1]$ según la codificación).

Esto tiene una **ventaja**: lo que alimenta la última regresión ya no son las entradas crudas, sino el vector de activaciones de la capa previa; en la práctica, **toda la capa de regresiones logísticas cumple el papel de un nuevo vector de características** (un “ x ” intermedio) para la neurona final.

Importante: **no** existe una relación lineal directa entre la salida de la última neurona y las **cinco entradas** iniciales (o, en general, las entradas originales): la composición de capas introduce **no linealidad**.

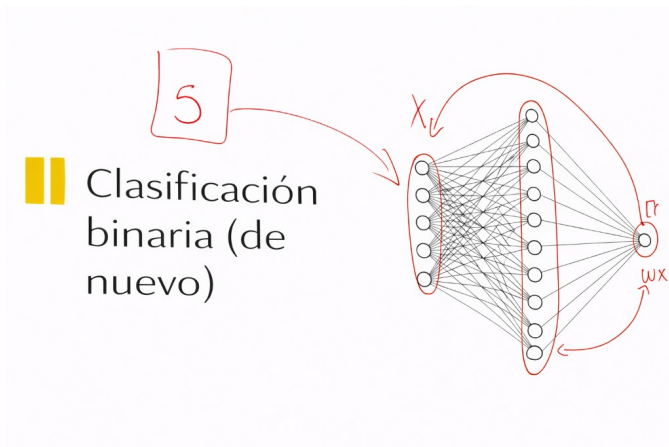


Figura 15. Clasificación binaria (de nuevo): entrada (ej. imagen o *features*), capa oculta con diez regresiones logísticas y salida única; la relación entrada-salida ya no es lineal de punta a punta.

Ahora se comenta la organización por **capas**:

- **Capa de entrada:** es la primera; recibe los datos (píxeles, *features*, etc.).
- **Capas intermedias (ocultas):** se pueden añadir **tantas como se necesite** (n capas), ampliando la red y la capacidad de representación respecto a una sola capa oculta.
- **Capa de salida:** debe tener un **número de salidas acorde al problema**. Por ejemplo, si el objetivo es clasificar **diez** clases u objetos, la capa de salida requiere $n = 10$ neuronas (o equivalente según la codificación elegida).

Redes neuronales.

- La **no linealidad** permite abordar **problemas complejos** que un modelo puramente lineal no separa bien.
- Están **organizadas en capas**; el número de capas y neuronas son **hiperparámetros** que se eligen al diseñar la arquitectura.
- Lo relevante es que **cada capa sea diferenciable** (o que la composición lo sea donde se entrena con gradientes).
- Si la función de pérdida y la red son derivables, se puede aplicar **optimización basada en gradiente** (p.ej., descenso).
- En **cada capa** hay **neuronas** (unidades que computan la transformación afín y, salvo la entrada, suelen llevar activación).

En buena medida, **ese es el secreto del *deep learning***: componer muchas transformaciones no lineales entrenables por gradiente. Por eso este tipo de arquitecturas se han ido aplicando a **problemas muy difíciles** (visión, lenguaje, señales, etc.), donde los modelos lineales o poco profundos no alcanzan.

Cada círculo del diagrama representa **neuronas**. Un ejemplo típico es: **una capa de entrada, dos capas intermedias** (ocultas) y **una capa de salida**. La figura 16 ilustra el

encadenamiento de matrices de pesos $W_0, W^1, \dots, W^n$ entre capas densas.

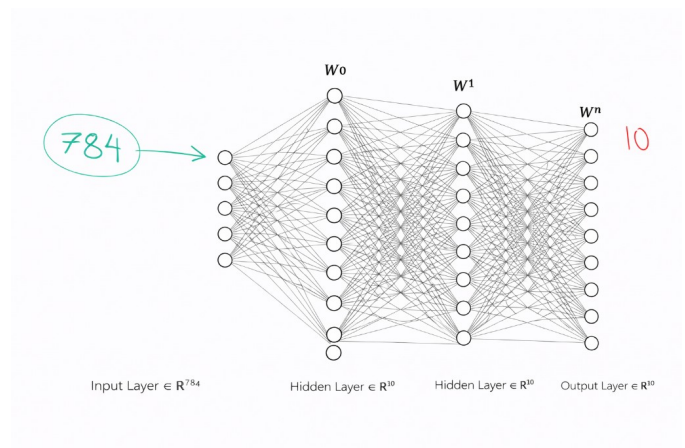


Figura 16. Red densa: entrada $\mathbb{R}^{784}$ (p.ej. MNIST aplanado), dos capas ocultas $\mathbb{R}^{10}$ y salida $\mathbb{R}^{10}$ para diez clases; los nodos dibujados son esquemáticos y las etiquetas indican las dimensiones reales.

V. ANUNCIOS

- Para el **jueves**, la tarea es **revisar el proyecto**.
- La **semana siguiente** será **asíncrona**: el profesor irá subiendo los temas o materiales correspondientes.